



2^{ème} partie

JavaScript

langage objet

Jean-Christophe DUBOIS
jean-christophe.dubois@univ-rennes.fr



Objet

- Définition

Un objet est une instantiation d'une Classe.

Élément, éventuellement complexe, défini par:

- un **constructeur** : création d'un nouvel objet.
Il définit toutes les caractéristiques initiales de l'objet
- des **propriétés** : variables rattachées à l'objet pour le définir
- des **méthodes** : fonctions utilisées pour effectuer des opérations sur les propriétés de l'objet

Un objet peut être abstrait : une date, une commande, un prix
ou concret : un produit, un client, une voiture

Création d'un Objet

- Création d'un objet et définition de ses propriétés

```
let monObjet = new Object() ;  
monObjet.prop1 = "texte";      // propriété textuelle  
monObjet.prop2 = 10;           // propriété numérique  
monObjet.prop3 = true;         // propriété booléenne
```

- Autre méthode, création d'un objet littéral

```
let monObjet = {  
  prop1: "texte",              // propriété textuelle  
  prop2: 10,                   // propriété numérique  
  prop3: true                  // propriété booléenne  
}  
console.log(typeof monObjet);  // "object"
```

*Rq: En réalité, **monObjet** est une **variable** qui contient un **objet***

Modification d'un Objet

- Création d'un objet vide

```
let monObjet = { };
```

- Ajout/Modification d'une propriété

```
monObjet["prop4"] = 1789;    // méthode par crochet  
monObjet.prop5 = "valeur1"; // méth. accesseur / point
```

La 2^{ème} méthode est un raccourci de la 1^{ère}

- Suppression d'une propriété

```
delete monObjet.prop5;
```

- Accès à une propriété

```
let val1 = monObjet.prop4;    // val1 vaut 1789
```

Propriétés d'un Objet

- Accès aux valeurs des propriétés d'un objet

```
let monObjet = {  
  prop1: "texte",           // propriété textuelle  
  prop2: 10                 // propriété numérique  
}  
console.log(Object.values(monObjet)) ;           // ["texte", 10]  
console.log(Object.values(monObjet)[0]);         // "texte"
```

- Accès aux intitulés des propriétés d'un objet

```
console.log(Object.keys(monObjet))               // ["prop1", "prop2"]  
console.log(Object.keys(monObjet)[0]);           // "prop1"
```

Propriétés d'un Objet

- Accès aux paires intitulés/valeurs d'un objet

```
let monObjet = {  
    prop1: "texte",           // propriété textuelle  
    prop2: 10                 // propriété numérique  
}  
console.log(Object.entries(monObjet)) ;  
           // [["prop1 ", "texte"], ["prop2", 10]]  
console.log(Object.entries(monObjet)[0]);  
           // ["prop1 ", "texte"]
```

- Vérification de l'existence d'une propriété d'un objet

```
console.log(monObjet.hasOwnProperty("prop1 "));    // true
```

Propriétés d'un Objet

- Variable utilisée comme propriété d'un objet
 - Soit la variable suivante :

```
const nomVal= "reponse";
```

- Le contenu de la variable nomVal peut être utilisé comme intitulé d'une propriété de l'objet

```
let monObjet = {  
  prop1: "texte",           // propriété textuelle  
  prop2: 10,                // propriété numérique  
  [nomVal]: true            // propriété booléenne  
}  
console.log(monObjet.reponse) ;           // true
```

Imbrication d'Objets

- Un objet peut lui-même avoir pour propriété un autre objet

```
let monObjet = {  
  prop1: {  
    unePropA : "valA",  
    unePropB : "valB"  
  },  
  prop2: 10,  
  prop3: true  
}  
console.log(typeof monObjet.prop1);    // "object"  
console.log(monObjet.prop1.unePropB);  // "valB"
```


Méthodes d'un Objet

- Un objet peut posséder ses propres fonctions

```
let monObjet = {  
  prop1: 10,  
  affiche: function() {  
    alert("Je suis un Objet !") ;  
  }  
}  
monObjet.affiche();
```

Annonce de la page <https://fiddle.jshell.net> :

Je suis un Objet !

OK

Opérateur this

- Une méthode peut accéder aux propriétés de l'objet

Le mot clé **this** fait référence à l'objet lui-même.

```
let monObjet = {  
  prop1: 10,  
  affiche: function() {  
    alert("Je ne suis pas le n° " + this.prop1  
    + " je suis un Objet libre !");  
  }  
}  
monObjet.affiche();
```

Annnonce de la page <https://fiddle.jshell.net> :

Je ne suis pas le n°10 je suis un Objet libre !

OK

Ajout d'une propriété/méthode

- Ajout d'une propriété/méthode à un objet déjà existant

```
let monObjet = {  
  prop1: 10,  
  affiche: function() {  
    alert("Je ne suis pas le n° " + this.prop1  
    + " je suis un Objet libre !");  
  }  
}  
monObjet.prop2 = "nouvelleProp" ;      // propriété ajoutée  
monObjet.presente = function() {  
  alert(" N° " + this.prop1 + " avec une " + this.prop2);  
}                                         // méthode ajoutée  
monObjet.presente();                    // N° 10 avec une nouvelleProp
```

Class et constructor (*syntaxe ES6*)

- Classe avec constructeur, propriétés et méthodes

```
class Planete {                                // nouvelle déclaration
    constructor (n, r) {
        this.nom = n;
        this.rayon = r;
    }
    toString() { return `${this.nom}
                        a un rayon de ${this.rayon}km` }
}
let mars = new Planete("Mars", 3389);          // même usage
console.log(mars.toString()); // Mars a un rayon de 3389km
```

- Contrairement aux fonctions, les classes doivent **obligatoirement** être définies **AVANT** de pouvoir être utilisées

Getter/Accesneur

- Get : Accès au contenu d'une propriété

```
class Planete {  
    constructor (n, r) {  
        this._nom = n;           // nom est renommé _nom  
        this._rayon = r;         // rayon est renommé _rayon  
    }                             // pour éviter la confusion !  
    get nom() { return this._nom } // définition du getteur  
}  
let mars = new Planete("Mars", 3389);  
console.log(mars.nom);           // appel du getteur sans ()  
                                 // Mars
```

Une **méthode** ne peut pas avoir le même nom qu'une **propriété**

Getter/Accesneur

- Accès au contenu d'une "propriété" calculée

```
class Planete {  
  constructor (n, r) {  
    this._nom = n;  
    this._rayon = r;  
  }  
  get perimetre() { // calcul de la propriété  
    return 2 * Math.PI * this._rayon }  
}  
let mars = new Planete("Mars", 3389);  
console.log(mars.perimetre + "km"); // appel du getteur  
// 21293.715km
```

Setter/Mutateur

- Set : modification du contenu d'une propriété

```
class Planete {  
    constructor (n, r) {  
        this._nom = n;  
        this._rayon = r;           // _rayon ≠ rayon !  
    }  
    set rayon(n) { this._rayon = n } // définition du setteur  
}  
let mars = new Planete("Mars", 3389);  
mars.rayon = 3390;                // modification de _rayon
```

Une **méthode** ne peut pas avoir le même nom qu'une **propriété**

Propriété statique

- Static : définition d'une propriété de classe

```
class Planete {  
    static systeme = "solaire" // propriété commune à toutes  
                                // les instances de Planete  
    constructor (n, r) {  
        this._nom = n;  
        this._rayon = r;  
    }  
}  
let mars = new Planete("Mars", 3389);  
console.log(Planete.systeme); // Solaire
```

Les propriétés **statiques** sont appelées **exclusivement** par une classe

Méthode statique

- **Static** : définition d'une méthode de classe

```
class Planete {  
    constructor (n, r) {  
        this.nom = n;  
        this.rayon = r;  
    }  
    static plusGrande( p1, p2) {    // comparaison de 2 planètes  
        return (p1.r > p2.r ? p1 : p2) }  
}  
let mars = new Planete("Mars", 3389);  
let venus = new Planete("Venus", 6051);  
console.log(Planete.plusGrande(mars,venus).nom);    // Venus
```

Les méthodes **statiques** sont appelées **exclusivement** par une classe

Extends, super...

- Héritage et appel au super constructeur

```
class PlaneteGaz extends Planete {  
    constructor (n, r, g) {  
        super (n, r);           // appel au constructeur de Planete  
        this.gaz = g;  
    }  
}  
let jupiter = new PlaneteGaz("Jupiter", 69911, "H");  
console.log(jupiter.toString()); // toString() de Planete  
                                // Jupiter a un rayon de 69911km
```

PlaneteGaz est une **sous-classe** de la **super classe** Planete

Extends, super...

- Appel d'une méthode parente

```
class PlaneteGaz extends Planete {
    constructor (n, r, g) {
        super (n, r);
        this.gaz = g;
    }
    toString() { return `${super.toString()} // toString() de Planete
        et est composée de ${this.gaz} ` }
}
let jupiter = new PlaneteGaz("Jupiter", 69911, "H");
console.log(jupiter.toString()); // toString() de PlaneteGaz
// Jupiter a un rayon de 69911km
// et est composée de H
```

Décomposition d'un objet

- Transformation des propriétés d'un objet en variables

```
let maPlanete = {  
  nom: "Venus",  
  rayon: 6051  
}  
let nom = maPlanete.nom;           // syntaxe  
let rayon = maPlanete.rayon;       // fastidieuse
```

En entourant les propriétés de l'objet avec des accolades `{ }`, les **variables** seront **créées** et les **valeurs affectées**.

```
let { nom, rayon } = maPlanete;    // écriture raccourcie  
console.log(nom);                  // Venus  
console.log(rayon);                // 6051
```

L'objet original **maPlanete** n'est pas affecté

Décomposition d'un objet

- Renommage de **variable** lors de la transformation des propriétés avec :

```
let maPlanete = {  
  nom: "Venus",  
  rayon: 6051  
}  
let { nom: nomP, rayon } = maPlanete;  
// nomP sera utilisé à la place de nom  
alert(nomP); // Venus
```

Opérateur in

- L'opérateur **in** teste l'existence d'une propriété :
 - retourne **true** si la propriété existe
 - **false** sinon

```
alert( "rayon" in maPlanete);           // true  
alert( "couleur" in maPlanete);         // false
```

- **Parcours des propriétés** avec une boucle **for ... in**

```
for (let valeur in maPlanete)  
    alert(maPlanete[valeur]);  
// Affichages successifs de Venus et de 6051
```

Imbrication d'Objets

- Accès aux propriétés d'un objet et à leurs propres propriétés

```
let monObjet = {  
  prop1: {    propA : "valA",  
             propB : "valB" },  
  prop2: {    propC : "valC",  
             propD : "valD" }  
}  
for (let prop in monObjet) {  
  console.log (prop);           // prop1 ... prop2  
  for (let ssProp in monObjet[prop])  
    console.log (ssProp);       // propA, propB ...  
                                 // propC, propD  
}
```

Valeurs et intitulés des propriétés

- **Exercice** : Lister :
 - la **valeur** de la première propriété
 - le **nombre** de propriétés d'*unePersonne*
 - toutes les **valeurs** d'*unePersonne*
 - toutes les **propriétés** d'*ouvrages*
 - la **profession** d'*unePersonne*
 - les **intitulés** des propriétés une à une
 - les **valeurs** des propriétés une à une

```
var unePersonne = {  
  prenom: "John",  
  nom: "Deuff",  
  profession: "Humoriste",  
  ouvrages: { livre1 : "L'Œuf au riz", livre2 : "Quoi de n'Œuf ?" }  
};
```


JSON

- JSON (JavaScript Object Notation) est un format de données qui permet de stocker et échanger des informations de manière structurée
- Les données sont organisées sous la forme d'une paire : "clé" : valeur
- Les valeurs des fichiers JSON (.json) peuvent contenir des chaînes de caractères "...", des nombres, des booléens, des objets {...}, des tableaux [...] ou encore des valeurs null.

```
{  
  "nom" : "Jupiter",  
  "lunes": ["Europe", "Ganymède", "Io"],  
  "attributs": { "rayon": 69911,  
                 "tellurique": false,  
                 "gravité": null }  
}
```

JSON

- JSON (JavaScript Object Notation) est un format de données qui permet de stocker et échanger des informations de manière structurée
- Les données
- Les valeurs caractères "tableaux [...], des

En JSON
{ "nom" : "Jupiter" }

En JavaScript
{ nom : "Jupiter" }

```
{  
  "nom" : "Jupiter",  
  "lune" : true,  
  "attributs": { "rayon": 69911,  
                 "tellurique": false,  
                 "gravité": null }  
}
```

JSON

- `parse()` : conversion d'une chaîne JSON (délimitée par des ') en objet JS

```
const txt = '{      "nom" : "Jupiter",
                    "lunes": ["Europe", "Ganymède", "Io"],
                    "attributs": { "rayon": 69911,
                                    "tellurique": false,
                                    "gravité": null }
                }';
let planete = JSON.parse(txt);
console.log(planete.nom);           // Jupiter
console.log(planete.lunes[2]);      // Io
console.log(planete.attributs.rayon); // 69911
```

JSON

- **stringify()** : conversion d'un objet JS en chaîne JSON

```
const objet = {    nom : "Jupiter",
                  lunes: [ "Europe", "Ganymède", "Io"],
                  attributs : {    rayon: 69911,
                                  tellurique: false,
                                  gravité: null }
};
const planeteJson = JSON.stringify(objet);
console.log(planeteJson);
// {"nom": "Jupiter", "lunes": ["Europe", "Ganymède", "Io"],
  "attributs": {"rayon": 69911, "tellurique": false, "gravité": null} }
```